



Иван Прасков
Senior Software Engineer @ DSKBank

Test what matters: Real testing with Testcontainers

Партньори:



Следете актуалните обяви за **.NET**





About me:

- Around 5 years of experience in various projects - legacy and greenfield cloud-based systems, microservices and monoliths
- Worked with .NET, C++, React, Angular
- Service and product companies



Agenda:

1. What are bad tests?
2. What is a “unit” in a unit test?
3. What should we test?
4. How does heavy mocking hurt our tests?
5. How Testcontainers help avoid that?
6. Demo
7. Q&A



What are bad tests?

- Fragile tests
- Fake-positive tests
- Slow tests
- Tests that test implementation details
- Duplicate tests



```
// .. Setup code
// ..
var actualCreditDecision = await _sut.MakeCreditRequestDecision(new(customer.CustomerId, 10_000), CancellationToken.None);
Assert.NotNull(actualCreditDecision);
```

Does not assert anything useful, can rarely fail

```
// .. Setup code
// ..
var actualCreditDecision = await _sut.MakeCreditRequestDecision(new(customer.CustomerId, 10_000), CancellationToken.None);
Assert.IsType<CreditRequestDecisionResponse>(actualCreditDecision);
```

In most cases will never fail, unless exception is thrown

```
// .. Setup code
// ..
var actualCreditDecision = await _sut.MakeCreditRequestDecision(new(customer.CustomerId, 10_000), CancellationToken.None);
Assert.True(actualCreditDecision!.MaxCreditAmount > 0);
Assert.NotEmpty(actualCreditDecision!.CreditRequestDecision);
```

False-positive in most cases, confirms nothing about the correct behavior

```
// .. Setup code
// ..
await _sut.MakeCreditRequestDecision(new(customer.CustomerId, 10_000), CancellationToken.None);

_customerRepositoryMock.Verify(x => x.GetCustomerById(It.IsAny<string>()), Times.Once);
_employmentHistoryClientMock.Verify(x => x.GetCustomerEmploymentHistory(It.IsAny<string>(), It.IsAny<CancellationToken>()), Times.Once);
_creditHistoryRepositoryMock.Verify(x => x.GetCustomerCreditHistory(It.IsAny<string>()), Times.Once);
_creditRequestRepositoryMock.Verify(x => x.SaveCreditRequest(It.IsAny<CreditRequest>()), Times.Once);
```

Tightly coupled to current implementation, fragile

```
// .. Setup code
// ..
await _sut.MakeCreditRequestDecision(new("12345123", 10_000), CancellationToken.None);
_loggerMock.Verify(static x =>
    x.Log(It.Is<It.IsAnyType>((v, _) => v.ToString().Contains("No customer found for command"))), Times.Once);
```

Doesn't test anything, fragile, logging is not part of the behavior



What is “unit” in unit test?

Unit = any isolated piece of code you decide to test :)

As small as you can reach your desired level of confidence

Can be a method, class, whole module, feature, endpoint

It is NOT just the smallest possible thing you can test (method, line of code)



What should we test?

1. Behavior, not implementation
2. Features in isolation
3. Features with clear requirements

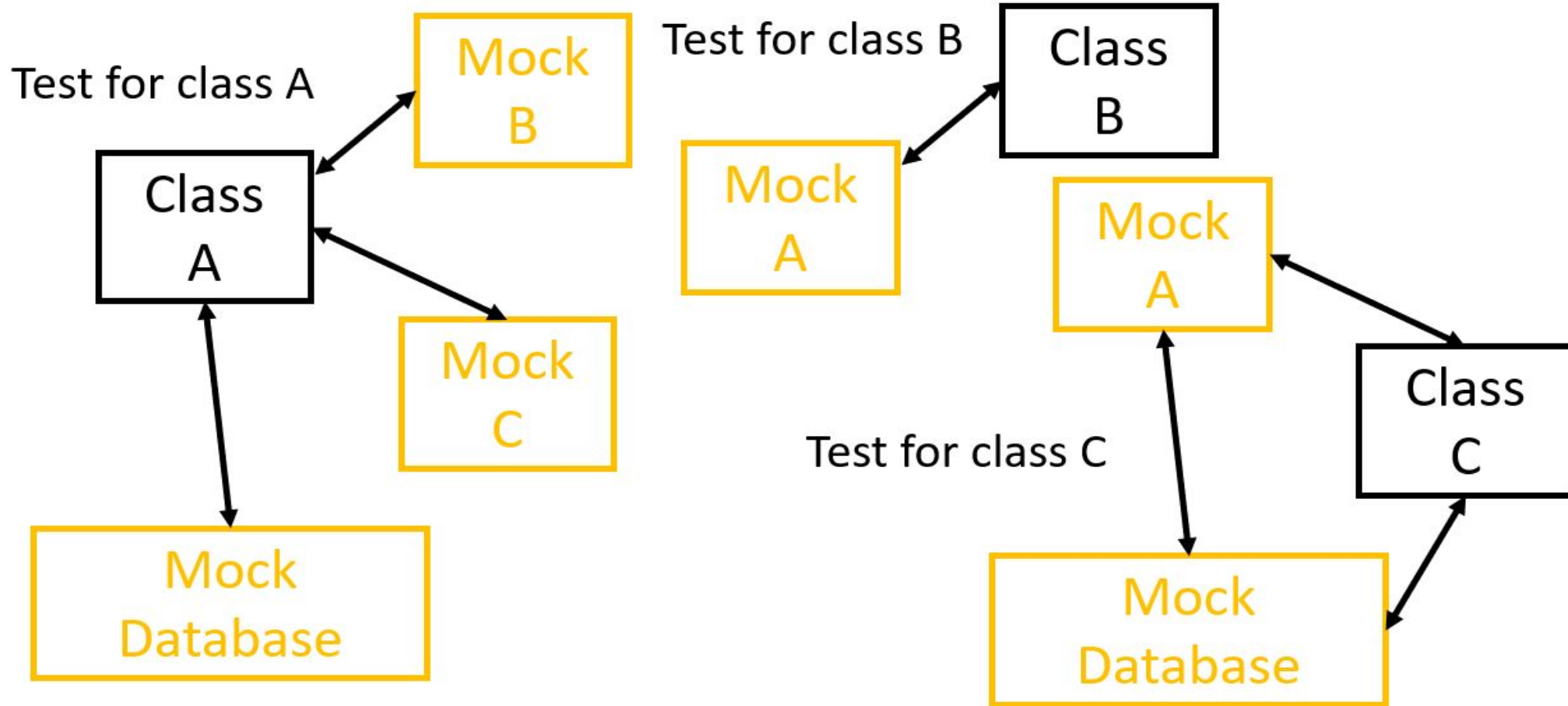


How does heavy mocking hurt our tests?

You can't refactor (easily)

Heavy mocking of all dependencies results in fragile tests

Mocking, requires making assumptions on how the code behaves.



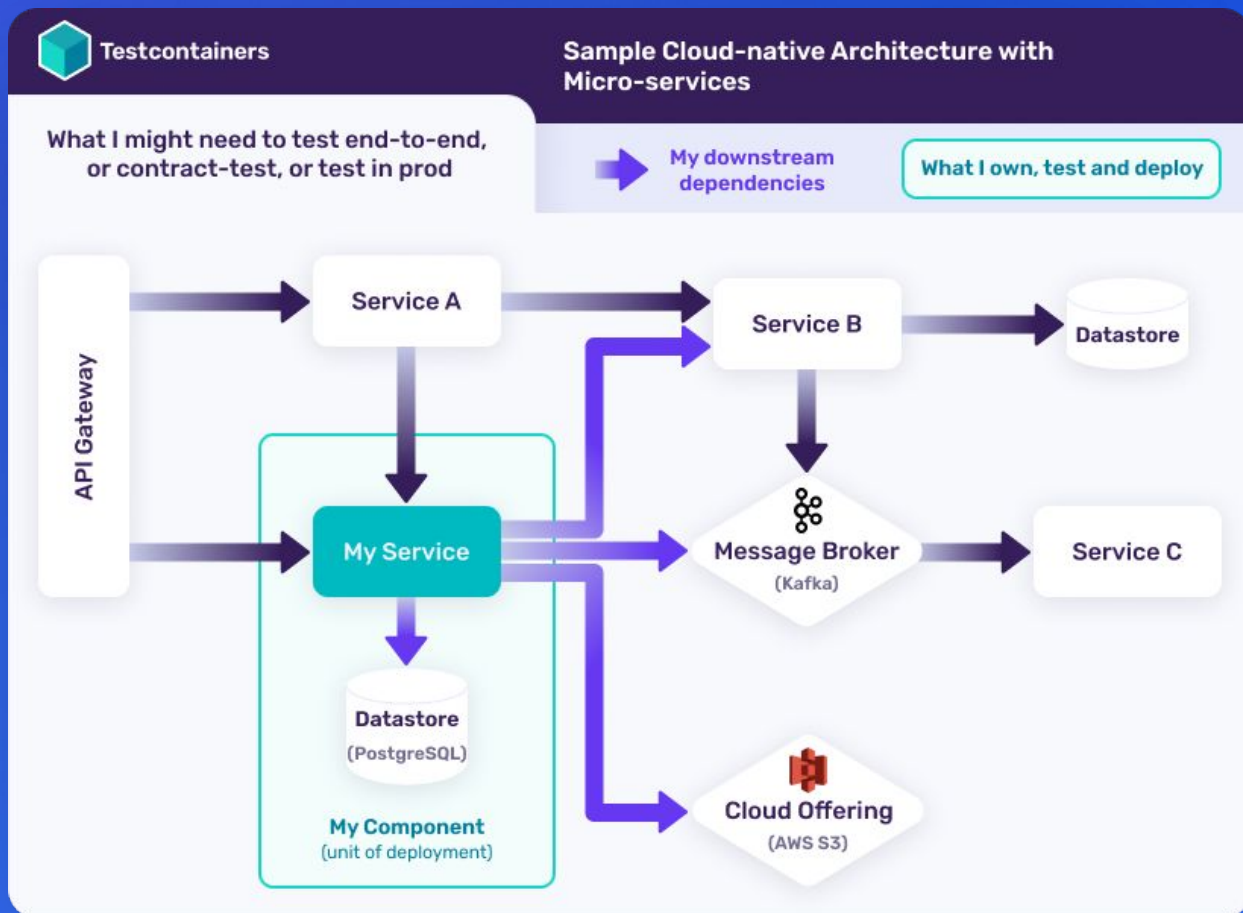
[Source](#)



Refactoring

Maintaining a clean codebase
requires refactoring confidently and doing it often
refactoring requires tests that don't break each time you change something
tests should test only behavior so that refactoring is quick

Testcontainers



Open source library

Provides lightweight, throwaway instances of:

- Databases
- Message brokers
- Browsers
- ..many more

Easy setup of dependencies as code

Automatically cleaned up

Only requires Docker



Demo

Credit scoring system

Sample system that calculates credit score, approves/rejects customer credit requests

3-part demo showcasing refactoring of a feature and its effect on different kinds of tests

Requirements (simplified)

Max credit score - **100**

Minimum approval score - **50**

Score between **50 and 60** - Manual approval required

Over **60** - Approved

Max credit amount, based on net monthly income (NMI):

score 80 and above: NMI **x 20**

score 70-80: NMI **x 10**

score 60-70: NMI **x 5**

score 50-60: NMI **x 3**

under 50: **0 (rejected)**

Score **penalties** based on:

Credit history (last 12m) and age:

No credit history: **-5**

1-2 missed payments: **-10**

3 or more: **-30**

Debt-to-income ratio:

30% and below: no penalty

30%-60%: **-20**

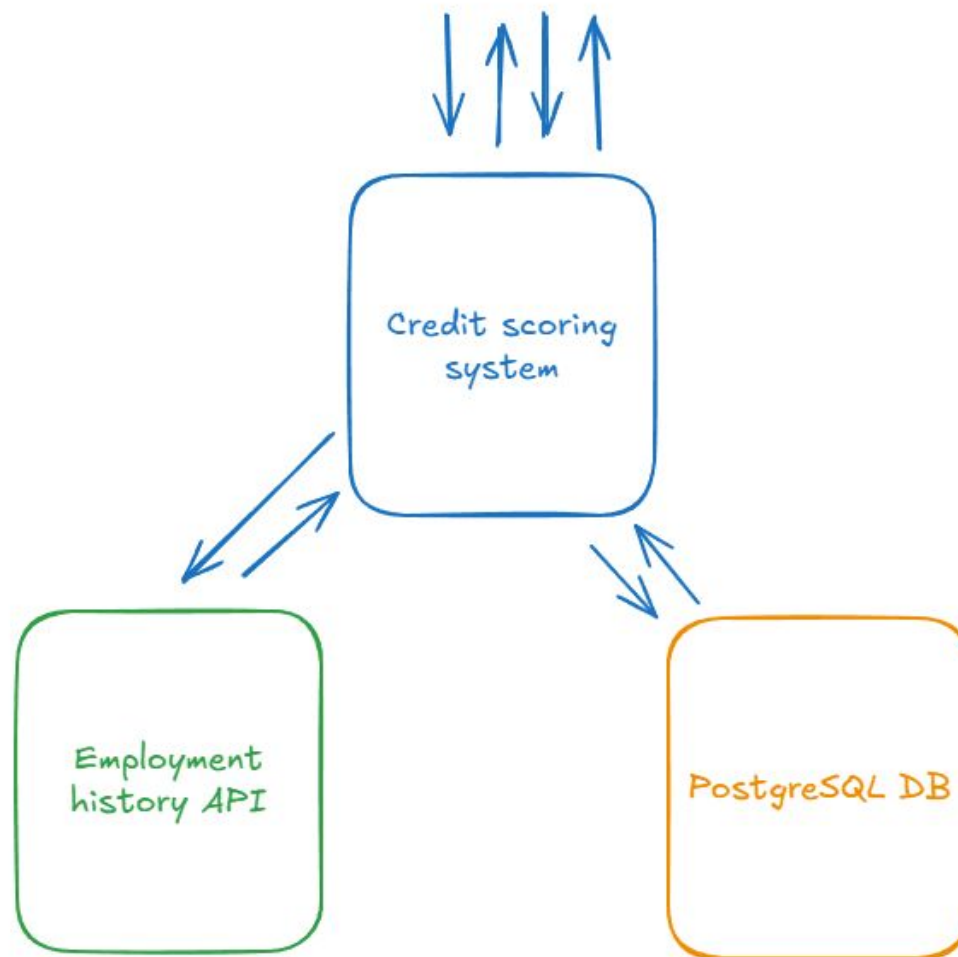
above 60%: **-40**

Score **bonus** based on employment stability:

5y+: **+15**

2-5y: **+10**

less than 1y: **0**



[Source](#)



Demo

Credit scoring system

Sample system that calculates credit score, approves/rejects customer credit requests

3-part demo showcasing refactoring of a feature and its effect on different kinds of tests

Requirements (original)

Max credit score - **100**

Minimum approval score - **50**

Score between **50 and 60** - Manual approval required

Over **60** - Approved

Max credit amount, based on net monthly income (NMI):

score 80 and above: NMI **x 20**

score 70-80: NMI **x 10**

score 60-70: NMI **x 5**

score 50-60: NMI **x 3**

under 50: **0 (rejected)**

Score **penalties** based on:

Credit history (last 12m) and age:

No credit history and under 25y old: **-5**

1-2 missed payments: **-10**

3 or more: **-30**

Debt-to-income ratio:

30% and below: no penalty

30%-50%: **-10**

50%-60%: **-20**

60%-70%: **-30**

above 70%: **-40**

Score **bonus** based on employment stability:

Full-time :

3y+: **+10**

1-3y: **+5**

less than 1y: **0**

Part-time:

2y+: **+5**

less than 2y: **0**

less than 1y:

Self-employed:

5y+: **+15**

2-5y: **+10**

less than 2y: **0**



TODOs

Add tests to cover all the uncovered cases, based on the provided requirements

Use EF instead of Dapper

Add endpoint to return only credit requests for Manual approval (score 50-60)

Add endpoint to manually approve/reject a credit request

On approval return multiple credit offers with different interest rates and periods

Add eventing to notify other service/s once credit request is approved and try to test this

Thank you!

Contacts:

in [Linkedin profile](#)

 [Github profile](#)

Resources:

Demo @
<https://github.com/IvanPraskov/RealTestingWithTestcontainers/>

Testcontainers:
<https://testcontainers.com/>

Repo showcasing arch evolution:
<https://github.com/evolutionary-architecture/evolutionary-architecture-by-example>

Books:

[Working effectively with Legacy code](#)

[TDD by Example](#)

[Refactoring](#)